

Real-Time Ai Prediction Platform With Automated Mlops Pipeline

1. Introduction

Artificial Intelligence and Machine Learning systems are increasingly used in modern enterprises to automate decision-making and predictive analysis. However, building a machine learning model alone is not sufficient for real-world production environments. Modern AI systems require deployment pipelines, monitoring systems, automated retraining mechanisms, and continuous integration workflows to maintain model performance and operational reliability.

This project implements a production-grade Real-Time AI Prediction Platform with an Automated MLOps Pipeline for customer churn prediction. The system was designed to simulate how enterprise-level AI platforms are developed, deployed, monitored, and continuously improved in real-world environments.

The platform integrates multiple MLOps components including:

- Real-time prediction API
- Automated model retraining
- MLflow experiment tracking
- Data drift monitoring
- Docker containerization
- CI/CD automation
- Cloud deployment
- Monitoring dashboard visualization

The complete workflow demonstrates the full lifecycle management of machine learning systems from data preprocessing to production deployment and monitoring.

2. Project Objectives

The primary objective of this project is to design and deploy an enterprise-style AI prediction platform capable of operating in real-time production environments.

The main objectives are:

- Build a real-time machine learning prediction API
- Automate model retraining workflows
- Implement continuous integration and deployment pipelines
- Track machine learning experiments using MLflow
- Detect data drift using monitoring techniques
- Deploy the system to a cloud platform

- Create a monitoring dashboard for visualization
- Simulate modern MLOps architecture used in enterprises

3. Technologies Used

Technology	Purpose
Python	Core Programming Language
Pandas	Data Processing and Manipulation
NumPy	Numerical Computation
Scikit-Learn	Machine Learning Model Development
FastAPI	Real-Time Prediction API
MLflow	Experiment Tracking
Docker	Application Containerization
GitHub Actions	CI/CD Automation
Streamlit	Monitoring Dashboard
Apache Airflow	Automated Retraining Pipeline

4. Dataset Description

The project uses the Telco Customer Churn dataset for predicting customer churn behavior.

The dataset contains customer-related information including:

- Customer demographics
- Subscription information
- Billing details
- Internet services
- Contract types
- Payment methods
- Churn labels

The target variable of the dataset is: Churn

The dataset was used to train a machine learning classification model capable of predicting whether a customer is likely to leave the service.

5. Data Preprocessing

Before training the machine learning model, data preprocessing operations were performed to improve data quality and model performance.

The preprocessing workflow included:

- Handling missing values
- Encoding categorical variables
- Feature selection
- Train-test splitting
- Data transformation and cleaning

Pandas and Scikit-Learn libraries were used extensively during preprocessing operations.

The preprocessing stage ensured that the dataset was suitable for machine learning model training and prediction generation.

6. Model Training

The machine learning model was developed using the Scikit-Learn framework.

The model training pipeline includes:

- Loading the dataset
- Preprocessing features
- Splitting training and testing data
- Training the machine learning model
- Evaluating model accuracy
- Saving the trained model

The trained model was serialized and saved as: `models/churn_model.pkl`

The training script used in this project is: `src/train.py`

The trained model achieved successful prediction performance and was later integrated into the FastAPI prediction service.

7. Mlflow Experiment Tracking

MLflow was integrated into the project to support experiment tracking and machine learning lifecycle management.

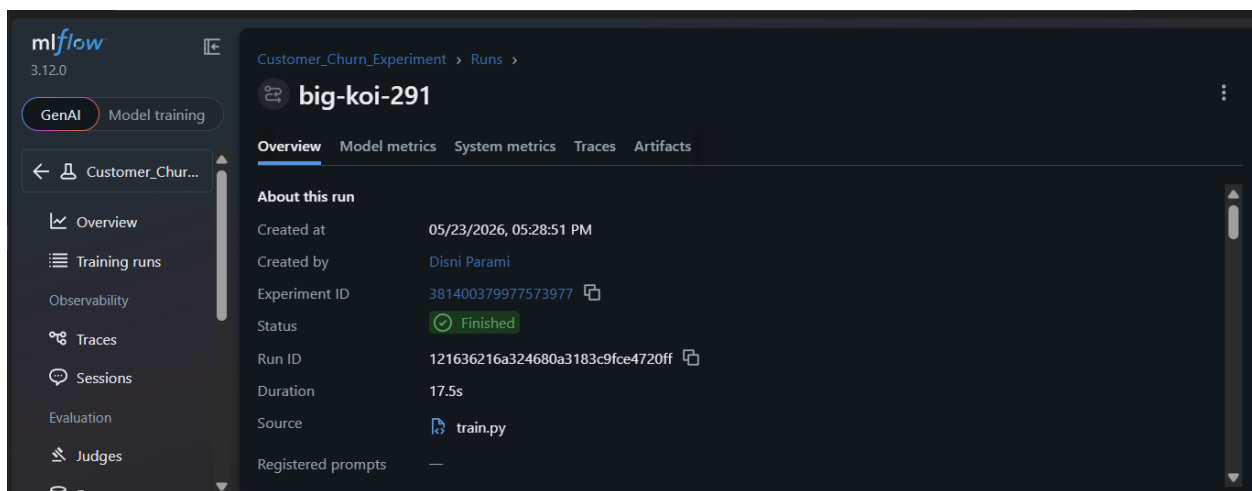
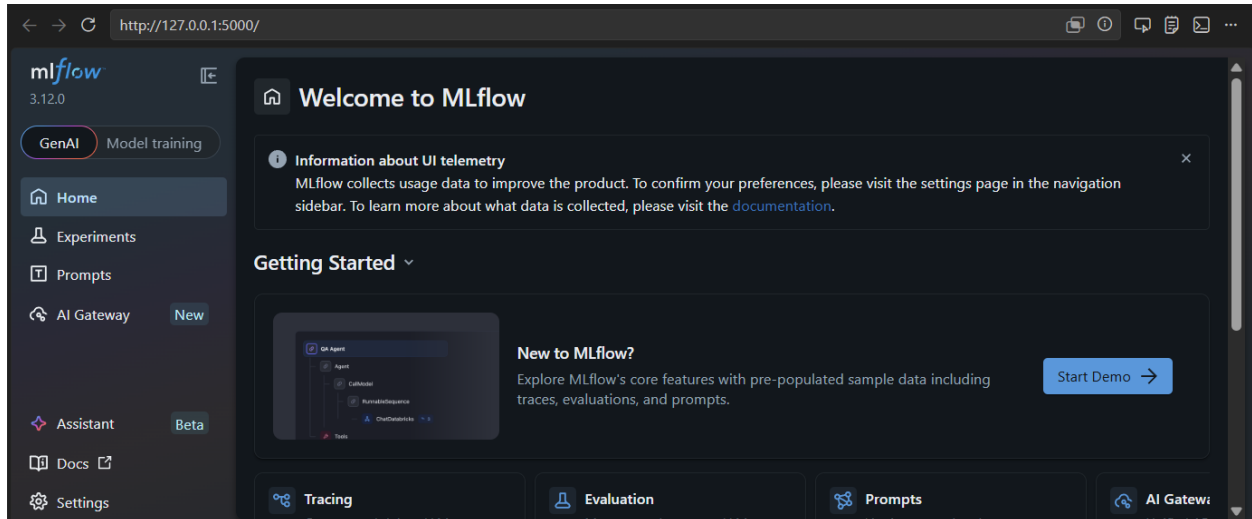
The following information was tracked using MLflow:

- Model accuracy metrics
- Training parameters
- Experiment runs

- Model artifacts
- Training history

MLflow provides a centralized interface to compare machine learning experiments and monitor model performance over time.

The MLflow UI dashboard allows users to visualize experiment runs and evaluate different model versions.



MLflow Dashboard

8. Fastapi Real-Time Prediction Api

FastAPI was used to develop a real-time prediction API for serving machine learning predictions.

The API provides:

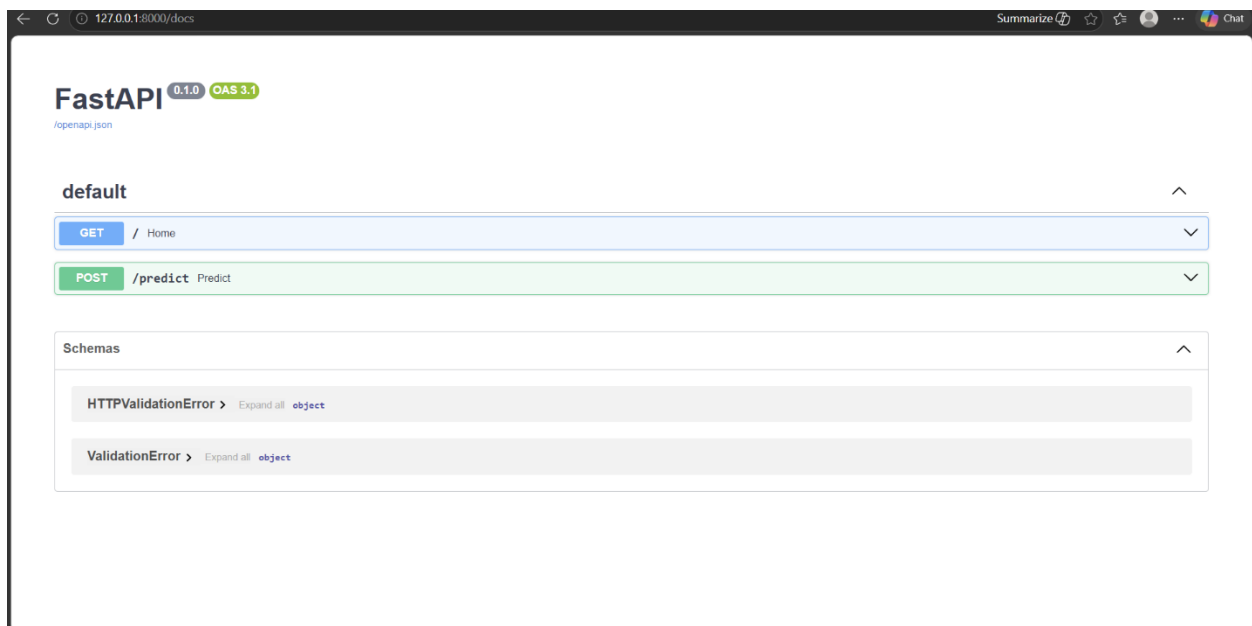
- Real-time prediction generation
- JSON request handling
- REST API endpoints
- Automatic Swagger documentation
- Fast and scalable performance

The primary API endpoint implemented is: /predict

The FastAPI application file is located at: api/app.py

The API accepts customer-related input data and returns churn prediction results in real-time.

Swagger UI was automatically generated for API testing and documentation.



Swagger UI



Prediction Result

9. Docker Containerization

Docker was used to containerize the FastAPI application and create a portable deployment environment.

Containerization provides several benefits including:

- Environment consistency
- Easier deployment
- Application portability
- Production-ready packaging
- Dependency isolation

The Docker container includes:

- FastAPI application
- Python dependencies
- Machine learning model
- Runtime configuration

Docker ensures that the application can run consistently across different systems and deployment environments.

10. CI/CD Pipeline Using Github Actions

GitHub Actions was integrated to automate CI/CD workflows within the project.

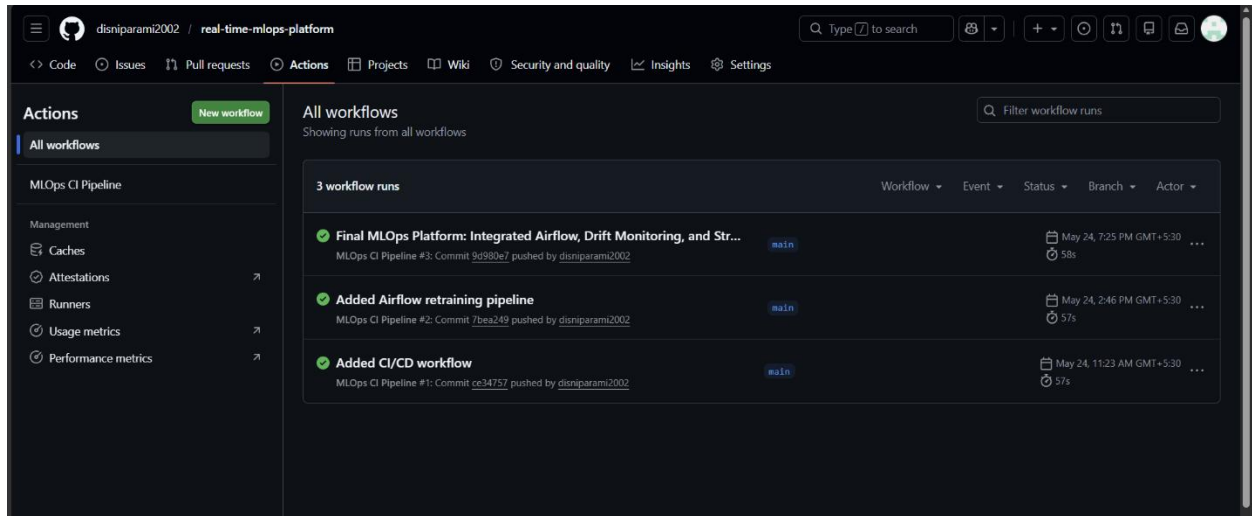
The CI/CD pipeline performs:

- Automated workflow execution
- Dependency installation

- Build validation
- Continuous integration checks

The workflow configuration file is located at: `.github/workflows/ci.yml`

GitHub Actions helps automate project workflows and improves software deployment reliability.



Github Actions

11. Data Drift Monitoring

Data drift monitoring was implemented to detect differences between incoming production data and the original training dataset.

The drift monitoring process includes:

- Statistical comparison
- Mean difference analysis
- Drift detection metrics
- Monitoring visualization

The drift monitoring script used in this project is: `src/drift_monitor.py`

Monitoring drift is important because machine learning models may lose prediction accuracy when incoming data changes significantly over time.

12. Automated Retraining Pipeline

An automated retraining pipeline was designed using Apache Airflow.

The retraining workflow performs:

- Scheduled retraining tasks
- Automated execution of training scripts
- Model updates
- Workflow orchestration

The Airflow pipeline configuration file is: `airflow/dags/retrain_pipeline.py`

The retraining pipeline simulates how enterprise AI systems continuously improve model performance without manual intervention.

13. Monitoring Dashboard

A Streamlit-based monitoring dashboard was developed for monitoring system behavior and model-related information.

The dashboard provides:

- Monitoring metrics
- Drift monitoring visualization
- Dataset visualization
- Model-related statistics
- Real-time monitoring support

The dashboard interface allows users to monitor system status and visualize important machine learning metrics interactively.

Real-Time AI Prediction Monitoring Dashboard

System Status

Model API Running

MLflow Tracking Active

Drift Monitoring Active

Sample Dataset

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	7590-A1fxTG	Female	0	Yes	No	1	No	No phone s
1	3375-GWDE	Male	0	No	No	24	Yes	No
2	3558-GP8H4	Male	0	No	No	2	Yes	No
3	7725-CICCN	Male	0	No	No	45	No	No phone s
4	9337-HQ7U	Female	0	No	No	2	Yes	No

Monthly Charges Statistics

MonthlyCharges	
count	7041
mean	54.7627
std	93.89
min	18.25
25%	39.5
50%	73.35
75%	81.85
max	118.75

Data Drift Monitoring

Training Mean: 64.7626924659915

Incoming Data Mean: 65.76140244000012

Difference: 1.0

No Significant Drift

MLOps Components

- ✓ Real-Time Predictions
- ✓ Automated Retraining
- ✓ CI/CD Integration
- ✓ Docker Deployment
- ✓ MLflow Experiment Tracking
- ✓ Data Drift Monitoring

Streamlit Dashboard

14. Cloud Deployment

The monitoring dashboard was deployed using Streamlit Community Cloud.

The live deployment URL is:

<https://real-time-mlops-platform-eeywujbeavtjvqb69sfwrk.streamlit.app/>

Cloud deployment allows the application to be accessed remotely through the internet without local installation requirements.

This deployment simulates real-world production hosting of AI monitoring systems.

15. System Architecture

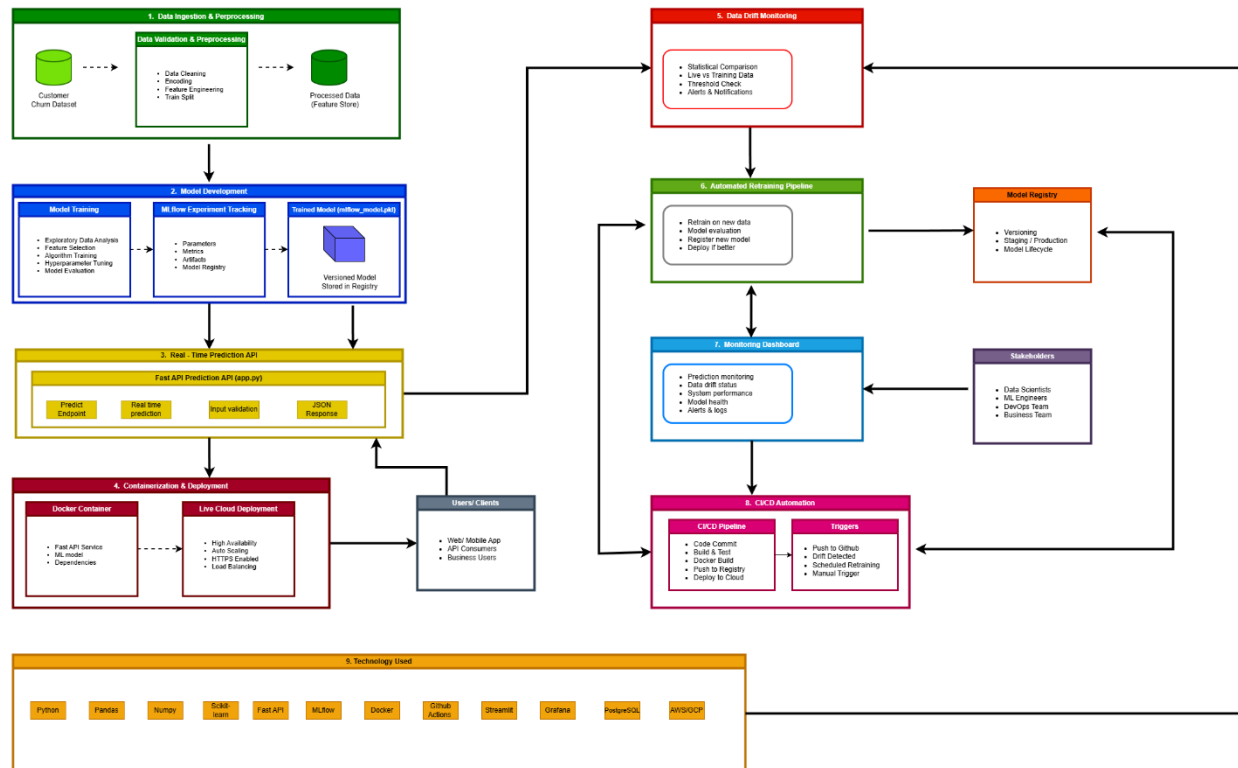
The system architecture represents the complete end-to-end MLOps workflow implemented in this project.

The architecture includes:

- Dataset ingestion
- Data preprocessing
- Model training
- MLflow experiment tracking
- FastAPI prediction service
- Docker containerization
- CI/CD automation
- Drift monitoring
- Automated retraining
- Monitoring dashboard
- Cloud deployment

The architecture demonstrates how modern enterprise AI systems are structured and maintained.

Customer Churn Prediction System



17. Challenges Faced

Several technical challenges were encountered during project development.

The main challenges included:

- Apache Airflow compatibility issues on Windows
- Dependency management conflicts
- MLflow configuration issues
- Docker environment setup challenges
- Free cloud deployment limitations

These issues were resolved through dependency configuration, environment setup adjustments, and workflow optimization.

18. Future Improvements

The following enhancements can be added in future versions of the system:

- Kubernetes deployment
- Real-time streaming pipelines
- Advanced drift detection algorithms
- Model rollback systems
- User authentication
- Database integration
- Advanced monitoring dashboards
- Automated alert systems

These improvements would further increase scalability, reliability, and enterprise readiness.

19. Conclusion

This project successfully demonstrates the implementation of a complete enterprise-style Real-Time AI Prediction Platform with an Automated MLOps Pipeline.

The system integrates machine learning engineering, deployment automation, monitoring, and cloud hosting into a unified workflow.

The platform successfully achieved:

- Real-time machine learning predictions
- Automated retraining
- Experiment tracking
- Data drift monitoring
- CI/CD automation

- Cloud deployment
- Dashboard monitoring

This project simulates how modern production AI systems are developed, deployed, monitored, and continuously maintained in enterprise environments.